

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №8

по дисциплине: Компьютерные сети
тема: «Программирование протокола HTTP»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей
Александрович

Проверил:
Рубцов Константин Анатольевич

Вариант 2.

Цель работы: изучить протокол HTTP и составить программу согласно заданию.

Теоретические сведения

HTTP (HyperTextTransferProtocol – протокол передачи гипертекста) – протокол прикладного уровня стека протоколов TCP/IP, предназначенный для передачи данных по сети с использованием транспортного протокола TCP. Текущая версия протокола HTTP v1.1, его спецификация приводится в документе RFC 2616.

Протокол HTTP может использоваться также в качестве «транспорта» для других протоколов прикладного уровня, таких как SOAP или XML-RPC.

Основой HTTP является технология «клиент-сервер». HTTP клиенты отсылают HTTP-запросы, которые содержат метод, обозначающий потребность клиента. Также такие запросы содержат универсальный идентификатор ресурса, указывающий на желаемый ресурс. Обычно такими ресурсами являются хранящиеся на сервере файлы. По умолчанию HTTP-запросы передаются на порт 80. HTTP сервер отправляет коды состояния, сообщая, успешно ли выполнен HTTP-запрос или же нет.

Основным объектом манипуляции в HTTP является ресурс, на который указывает URI (UniformResourceIdentifier) в запросе клиента. Обычно такими ресурсами являются хранящиеся на сервере файлы, но ими могут быть логические объекты или что-то абстрактное. Особенностью протокола HTTP является возможность указать в запросе и ответе способ представления одного и того же ресурса по различным параметрам: формату, кодировке, языку и т.д. Именно благодаря возможности указания способа кодирования сообщения клиент и сервер могут обмениваться двоичными данными, хотя данный протокол является текстовым.

Унифицированный идентификатор ресурса представляет собой сочетание унифицированного указателя ресурса (UniformResourceLocator, URL) и унифицированного имени ресурса (Uniform ResourceName, URN)

Например:

URI= http://iitus.bstu.ru/to_schoolleaver/230400

URN= to_schoolleaver/230400

URL= <http://iitus.bstu.ru/>

Метод протокола HTTP – это команда, передаваемая HTTP клиентом HTTP-серверу. В табл. 8.1. перечислены некоторые методы, определенные в протоколе HTTP v1.1. Полный список методов HTTP v1.1. содержится в документе RFC 2616.

Основные методы HTTP v1.1

Метод: GET

Назначение: Используется для запроса содержимого ресурса, на который указывает URI, содержащийся в запросе

Метод:HEAD

Назначение: Используется для извлечения метаданных или проверки наличия ресурса, на который указывает URI, содержащийся в запросе

Метод:POST

Назначение: Применяется для передачи данных заданному ресурсу. Данный метод предполагает, что по указанному URI будет производиться обработка передаваемого клиентом содержимого

Метод:PUT

Назначение: Применяется для передачи данных заданному ресурсу. Данный метод предполагает, что передаваемое клиентом содержимое соответствует находящемуся под данным URI ресурсу

Метод:OPTIONS

Назначение: Используется для определения возможностей HTTP сервера или параметров соединения для конкретного ресурса

Метод:DELETE

Назначение: Применяется для удаления ресурса, на который указывает URI

Каждый сервер обязан поддерживать как минимум методы GET и HEAD. Если сервер не распознал указанный клиентом метод, то он должен вернуть статус 501 (NotImplemented). Если серверу метод известен, но он неприменим к конкретному ресурсу, то возвращается сообщение с кодом 405 (MethodNotAllowed). В обоих случаях серверу следует включить в сообщение ответа заголовок Allow со списком поддерживаемых методов.

Код состояния HTTP представляет собой целое число из трех цифр. Первая цифра указывает на класс состояния:

- информационные сообщения;
- успешное выполнение;
- переадресация;
- ошибка клиента;
- ошибка сервера.

Каждое HTTP-сообщение состоит из трех частей, которые передаются в следующем порядке:

1. Стартовая строка – определяет тип сообщения;
2. Заголовки – характеризуют тело сообщения, параметры передачи и прочие сведения;
3. Тело сообщения – непосредственно данные сообщения.

Стартовые строки HTTP-сообщения различаются для запроса и ответа. Стартовая строка HTTP-запроса имеет следующий формат: метод URI HTTP/Версия, где метод - название запроса, URI определяет путь к запрашиваемому документу, версия - пара разделённых точкой арабских цифр.

Стартовая строка HTTP-ответа имеет следующий формат: HTTP/Версия КодСостояния Пояснение.

Заголовок HTTP представляет собой строку в HTTP-сообщении, содержащую разделённую двоеточием пару параметр-значение. Заголовки должны отделяться от тела сообщения хотя бы одной пустой строкой. Все

заголовки разделяются на четыре основных группы:

1. Основные заголовки (GeneralHeaders) – должны включаться в любое сообщение клиента и сервера.
2. Заголовки запроса (RequestHeaders) – используются только в запросах клиента.
3. Заголовки ответа (ResponseHeaders) – только для ответов от сервера.
4. Заголовки сущности (EntityHeaders) – сопровождают каждую сущность сообщения.

Тело HTTP-сообщения, если оно присутствует, используется для передачи данных, связанных с запросом или ответом. Чтобы понять, как работает протокол HTTP, рассмотрим пример получения

HTML-страницы с HTTP-сервера.

HTTP-запрос:

GET /to_schoolleaver/230400 HTTP/1.1

Host: iitus.bstu.ru

User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)

Accept: text/html

Accept-Language: ru

Connection: close

HTTP-ответ:

HTTP/1.1 200 OK

Date: Fri, 16 Dec 2012 13:45:00 GMT

Server: Apache

Last-Modified: Wed, 11 Feb 2009 11:20:59 GMT

Content-Language: ru

Content-Type: text/html; charset=utf-8

Content-Length: 1234

Connection: close

Протокол HTML позволяет достаточно легко создавать клиентские приложения. Возможности протокола можно расширить благодаря внедрению своих собственных заголовков, с помощью которых можно получить необходимую функциональность при решении нетривиальной задачи. При этом сохраняется совместимость с другими клиентами и серверами: они будут просто игнорировать неизвестные им заголовки.

Протокол HTTP устанавливает отдельную TCP-сессию на каждый запрос; в более поздних версиях HTTP было разрешено делать несколько запросов в ходе одной TCP-сессии, но браузеры обычно запрашивают только страницу и включённые в неё объекты (картинки, каскадные стили и т. п.), а затем сразу разрывают TCP-сессию. Для поддержки авторизованного (неанонимного) доступа в HTTP используются cookies; причём такой способ авторизации позволяет сохранить сессию даже после перезагрузки клиента и сервера.

Анализ применяемых функций

- **WSAStartup** - инициализирует использование Winsock DLL.
wVersionRequested - запрашиваемая версия Winsock,
lpWSAData - указатель на структуру **WSAData** для заполнения,
Возвращает: 0 при успехе, иначе код ошибки
 - **socket** - создает сокет для сетевого взаимодействия.
- af** - семейство адресов (**AF_INET** для IPv4),
type - тип сокета (**SOCK_STREAM** для TCP),
protocol - протокол (0 для выбора по умолчанию),
Возвращает: дескриптор сокета или **INVALID_SOCKET** при ошибке
 - **bind** - привязывает сокет к конкретному адресу и порту.
- s** - дескриптор сокета,
name - указатель на структуру **sockaddr** с адресом,
namelen - размер структуры адреса,
Возвращает: 0 при успехе, **SOCKET_ERROR** при ошибке
 - **listen** - переводит сокет в режим ожидания подключений.
- s** - дескриптор сокета,
backlog - максимальная длина очереди ожидающих подключений,
Возвращает: 0 при успехе, **SOCKET_ERROR** при ошибке
 - **accept** - принимает входящее подключение на сокете.
- s** - дескриптор сокета в режиме прослушивания,
addr - указатель на структуру для сохранения адреса клиента,
addrlen - размер структуры адреса,
Возвращает: дескриптор нового сокета для общения с клиентом
 - **recv** - получает данные из подключенного сокета.
- s** - дескриптор сокета,
buf - буфер для полученных данных,
len - размер буфера,
flags - флаги управления,
Возвращает: количество полученных байт, 0 при закрытии соединения,
SOCKET_ERROR при ошибке
 - **send** - отправляет данные через подключенный сокет.
- s** - дескриптор сокета,
buf - буфер с данными для отправки,
len - размер данных,
flags - флаги управления,
Возвращает: количество отправленных байт или **SOCKET_ERROR** при ошибке
 - **closesocket** - закрывает сокет.
- s** - дескриптор сокета,
Возвращает: 0 при успехе, **SOCKET_ERROR** при ошибке
 - **WSACleanup** - завершает использование Winsock DLL.
- Возвращает: 0 при успехе, иначе код ошибки
 - **htons** - преобразует 16-битное число из порядка хоста в сетевой.
- hostshort** - число в порядке хоста,
Возвращает: число в сетевом порядке байт

Листинги исходного кода:

```
#include <WinSock2.h>
#include <winsock.h>
#include <ws2tcpip.h>
#include <iostream>
#include <exception>
#include <string>
#include <iostream>
#include <fstream>
#include <chrono>
#include <vector>
#include <sstream>
// #include <pthread.h>

#define STR_BUF_SIZE 256

struct server_args
{
    SOCKET server_socket;
    sockaddr_in socket_address;
};

struct Response
{
    int code;
    std::string head;
    std::string body;
};

std::string stuck_response(Response r)
{
    return r.head + "\r\n\r\n" + r.body;
}

Response get_server_error_page()
{
    int code = 501;
    std::stringstream responseHead;
    std::stringstream responseBody;

    responseBody << "<!DOCTYPE HTML>"
        << "<html>"
        << "<head>"
        << "<title>" << std::to_string(code) << " Not
Implemented</title>"
        << "</head>"
        << "<body>"
        << "<h1>Not Implemented</h1>"
        << "<p>This request can not be processed by this server</p>"
        << "</body>"
        << "</html>";
```

```

    responseHead << "HTTP/1.1 " << std::to_string(code) << " Not
Implemented\r\n"
    << "Version: HTTP/1.1\r\n"
    << "Content-Type: text/html; charset=utf-8\r\n"
    << "Content-Length: " << responseBody.str().length();

    return {
        code,
        responseHead.str(),
        responseBody.str()};
}

```

Response get_ok_page_headers()

```

{
    int code = 200;
    std::stringstream responseHead;
    std::stringstream responseBody;

    responseHead << "HTTP/1.1 " << std::to_string(code) << " OK\r\n"
    << "Version: HTTP/1.1\r\n"
    << "Content-Type: text/html; charset=utf-8\r\n"
    << "Content-Length: "
    << responseBody.str().length()
    << "\r\n\r\n";

    return {
        code,
        responseHead.str(),
        responseBody.str()};
}

```

Response get_ok_page()

```

{
    int code = 200;
    std::stringstream responseHead;
    std::stringstream responseBody;

    responseBody << "<!DOCTYPE HTML>"
    << "<html>"
    << "<head>"
    << "<title>" << std::to_string(code) << " Good</title>"
    << "</head>"
    << "<body>"
    << "<h1>Hello world!</h1>"
    << "<p>Are you want see another?</p>"
    << "</body>"
    << "</html>";

    responseHead << "HTTP/1.1 " << std::to_string(code) << " OK\r\n"
    << "Version: HTTP/1.1\r\n"
    << "Content-Type: text/html; charset=utf-8\r\n"
    << "Content-Length: "
    << responseBody.str().length()

```



```

        << "\r\n\r\n";

    return {
        code,
        responseHead.str(),
        responseBody.str()};
}

Response get_client_error_page()
{
    int code = 404;
    std::stringstream responseHead;
    std::stringstream responseBody;

    responseBody << "<!DOCTYPE HTML>"
        << "<html>"
        << "<head>"
        << "<title>404 Not Found</title>"
        << "</head>"
        << "<body>"
        << "<h1> 404 </h1>"
        << "<p></p>"
        << "</body>"
        << "</html>";

    responseHead << "HTTP/1.1 " << std::to_string(code) << " Not Found\r\n"
        << "Version: HTTP/1.1\r\n"
        << "Content-Type: text/html; charset=utf-8\r\n"
        << "Content-Length: "
        << responseBody.str().length()
        << "\r\n\r\n";

    return {
        code,
        responseHead.str(),
        responseBody.str()};
}

bool should_run = false;
std::vector<SOCKET> clients;

void throw_error_with_code()
{
    std::string err_msg = "error with code: ";
    err_msg += std::to_string(WSAGetLastError());
    throw std::runtime_error(err_msg);
}

const char *getaddrbyname(const char *hostname)
{
    struct hostent *host_info;
    struct in_addr addr = {0};
    char *ip = NULL;

```

```

    if ((host_info = gethostbyname(hostname)) == NULL)
        return NULL;

    addr.s_addr = *(u_long *)host_info->h_addr_list[0];

    return inet_ntoa(addr);
}

void startup_wsa();
SOCKET get_socket_descriptor();
sockaddr_in get_bind_addr(const char *address, unsigned short port);
void bind_socket_with_address(SOCKET socket_descriptor, sockaddr_in bind_addr);
void listen_connections(SOCKET socket_descriptor);

SOCKET connect(const char *address, unsigned short port)
{
    std::clog << "start connect..." << std::endl;

    startup_wsa();

    std::clog << "WSA started..." << std::endl;

    SOCKET socket_descriptor = get_socket_descriptor();
    std::clog << "create socket" << std::endl;

    sockaddr_in bind_addr = get_bind_addr(address, port);
    std::clog << "create bind address" << std::endl;

    bind_socket_with_address(socket_descriptor, bind_addr);
    std::clog << "bind socket with address\nconnected" << std::endl;

    listen_connections(socket_descriptor);
    std::clog << "listen started" << std::endl;

    return socket_descriptor;
}

void startup_wsa()
{
    WORD wVersionRequested;
    WSADATA wsaData;
    wVersionRequested = MAKEWORD(2, 0);

    if (WSAStartup(wVersionRequested, &wsaData) == SOCKET_ERROR)
        throw_error_with_code();
}

SOCKET get_socket_descriptor()
{
    SOCKET res = socket(
        AF_INET,
        SOCK_STREAM,

```

```

    0);

    if (res == INVALID_SOCKET)
        throw_error_with_code();

    return res;
}

sockaddr_in get_bind_addr(const char *address, unsigned short port)
{
    sockaddr_in res;

    res.sin_family = AF_INET;
    res.sin_addr.s_addr = inet_addr(address);
    res.sin_port = htons(port);

    return res;
}

void bind_socket_with_address(SOCKET socket_descriptor, sockaddr_in bind_addr)
{
    if (bind(socket_descriptor, (sockaddr *)&bind_addr, sizeof(bind_addr)) ==
        SOCKET_ERROR)
        throw_error_with_code();
}

void listen_connections(SOCKET socket_descriptor)
{
    if (listen(socket_descriptor, 3) == SOCKET_ERROR)
        throw_error_with_code();
}

void handle_client(SOCKET client_socket);

void loop_accept_clients(SOCKET con)
{
    should_run = true;
    while (should_run)
    {
        sockaddr_in client_addr;
        int client_addr_size = sizeof(client_addr);
        std::cout << "waiting accept" << std::endl;
        SOCKET client_socket = accept(con, (sockaddr *)&client_addr,
&client_addr_size);

        if (client_socket != INVALID_SOCKET)
        {
            std::cout << "handle client" << std::endl;
            handle_client(client_socket);
        }
        else
            throw_error_with_code();
    }
}

```

```

}

////////////////////////////////////
////////////////////////////////////

void handle_client(SOCKET client_socket)
{
    char request_buffer[STR_BUF_SIZE];
    int request_buffer_len = STR_BUF_SIZE;
    int iResult;
    int iSendResult;
    char method[STR_BUF_SIZE];
    char URI[STR_BUF_SIZE];
    char host[STR_BUF_SIZE];
    int ver_h = 0;
    int ver_l = 0;

    iResult = recv(client_socket, request_buffer, request_buffer_len, 0);
    if (iResult > 0)
    {
        request_buffer[iResult] = '\0';
        sscanf(request_buffer, "%s %s HTTP/%i.%i\nHost: %s", method, URI,
&ver_h, &ver_l, host);
        printf("Client ask method %s for URI %s (HTTP ver %i.%i)\nHost: %s\n",
method, URI, ver_h, ver_l, host);
    }
    else if (iResult == 0)
    {
        printf("Close connection\n");
        return;
    }
    else
        throw_error_with_code();

    Response response = {};
    if (strcmp(URI, "/index.html"))
        response = get_client_error_page();
    else if (strcmp(method, "GET") == 0)
        response = get_ok_page();
    else if (strcmp(method, "HEAD") == 0)
        response = get_ok_page_headers();
    else
        response = get_server_error_page();

    std::string stucked_response = stuck_response(response);
    iSendResult = send(
        client_socket,
        stucked_response.c_str(),
        stucked_response.length(),
        0);
}

```

```

////////////////////////////////////
////////////////////////////////////

void disconnect(SOCKET connection)
{
    std::clog << "start disconnect..." << std::endl;

    if (closesocket(connection) == SOCKET_ERROR)
        throw_error_with_code();
    WSACleanup();

    std::clog << "disconnected" << std::endl;
}

////////////////////////////////////
////////////////////////////////////

int main()
{
    auto con = connect("127.0.0.1", 80);

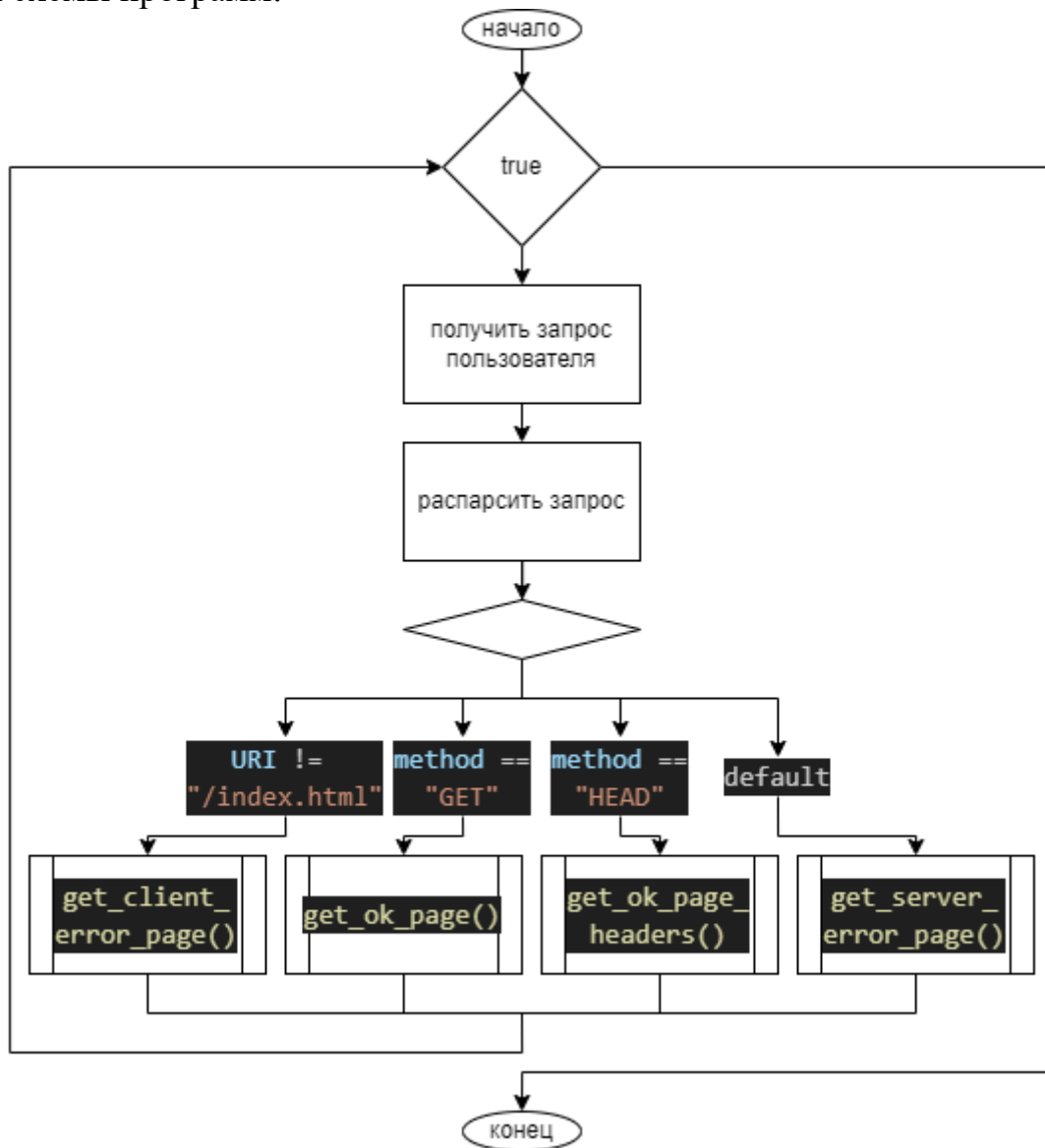
    std::clog << "start accept clients" << std::endl;
    loop_accept_clients(con);

    disconnect(con);

    return 0;
}

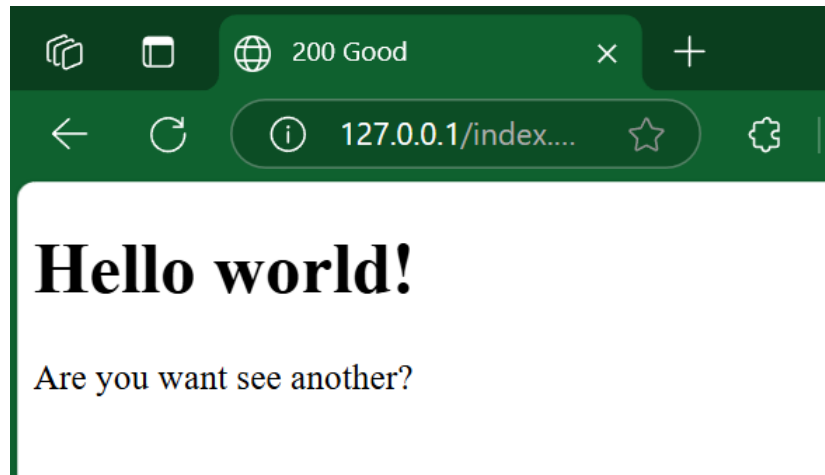
```

Блок схемы программ:

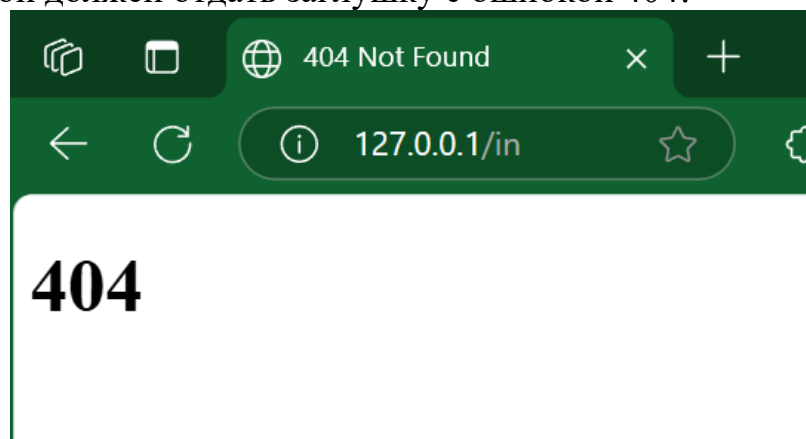


Анализ функционирования разработанных программ

Программа была протестирована с помощью браузера и программы postman. Переходя по адресу 127.0.0.1/index.html сервер отдаёт положительный ответ:



В случае, если серверу будет отправлен запрос на страницу, которой не существует, он должен отдать заглушку с ошибкой 404:



Также сервер обрабатывает команду HEAD по адресу 127.0.0.1/index.html.
Вывод из postman:

The screenshot shows a Postman interface with a HEAD request to 127.0.0.1/index.html. The response is a 200 OK status with a response time of 17 ms and a body size of 97 B. The response headers are displayed in a table:

Key	Value	Description
Version	HTTP/1.1	
Content-Type	text/html; charset=utf-8	
Content-Length	0	

Если указать любую другую команду по тому же адресу, то будет получена ошибка сервера. Вывод из postman:

The screenshot shows a Postman interface with a POST request to 127.0.0.1/index.html. The response is a 501 Not Implemented status with a response time of 13.24 s and a body size of 279 B. The response headers are displayed in a table:

Key	Value	Description
Version	HTTP/1.1	
Content-Type	text/html; charset=utf-8	
Content-Length	167	

The response body is shown in HTML format:

```
1 <!DOCTYPE HTML>
2 <html>
3
4 <head>
5   <title>501 Not Implemented</title>
6 </head>
7
8 <body>
9   <h1>Not Implemented</h1>
10  <p>This request can not be processed by this server</p>
11 </body>
12
13 </html>
```


Примерный вывод сервера выглядит следующим образом:

```
C:\Users\dmityr\Projects\BSTU_lab\3kurs2sem\КомпСети\lab8>main
start connect...
WSA started...
create socket
create bind address
bind socket with address
connected
listen started
start accept clients
waiting accept
handle client
Client ask method GET for URI /index.html (HTTP ver 1.1)
Host: 127.0.0.1
waiting accept
handle client
Close connection
waiting accept
handle client
Client ask method HEAD for URI /index.html (HTTP ver 1.1)
Host:
waiting accept
handle client
Client ask method GET for URI /index.html (HTTP ver 1.1)
Host:
waiting accept
```

Вывод: в ходе выполнения лабораторной работы был изучен протокол HTTP. HTTP (Hyper Text Transfer Protocol – протокол передачи гипертекста) – протокол прикладного уровня стека протоколов TCP/IP, предназначенный для передачи данных по сети с использованием транспортного протокола TCP. Протокол HTTP может использоваться также в качестве «транспорта» для других протоколов прикладного уровня. Метод протокола HTTP – это команда, передаваемая HTTP-клиентом HTTP-серверу.